NAME = ARAYANA KHARE

SAP ID = 590042432

BATCH = 15

Question 1: Find the Longest Word in a Sentence

1. Aim

To write a C program using a separate user-defined function to find the longest word in a given sentence.

2. Algorithm

1. Start the program.
2. Declare a character array to store the sentence.
3. Read the sentence using `fgets()`.
4. Call the `findLongestWord()` function.
5. Separate the sentence into words using `strtok()`.
6. Find the length of each word using `strlen()`.
7. Compare each word's length with the longest word found so far.
8. Store the longest word using `strcpy()`.

9. Display the longest word.

10. Stop the program.

3. C Program

```
#include <stdio.h>
#include <string.h>

#define SIZE 200

void findLongestWord(char sentence[], char longest[]);

int main()
{
    char sentence[SIZE];
    char longest[SIZE];

    printf("Enter a sentence: ");
    fgets(sentence, SIZE, stdin);

    sentence[strcspn(sentence, "\n")] = '\0';

    findLongestWord(sentence, longest);

    if (longest[0] == '\0')
        printf("No word found.\n");
    else
        printf("Longest word: %s\n", longest);

    return 0;
}

void findLongestWord(char sentence[], char longest[])
{
    char temp[SIZE];
    char *word;
    int maxLength = 0;

    strcpy(temp, sentence);
    longest[0] = '\0';
```

```
    word = strtok(temp, " \t");

    while (word != NULL)
    {
        if ((int)strlen(word) > maxLength)
        {
            maxLength = strlen(word);
            strcpy(longest, word);
        }

        word = strtok(NULL, " \t");
    }
}
```

4. Step-by-step explanation

Step 1: Declare the strings

```
char sentence[SIZE];
char longest[SIZE];
```

- `sentence` stores the complete sentence.

- `longest` stores the longest word.

Step 2: Read the sentence

```
fgets(sentence, SIZE, stdin);
sentence[strcspn(sentence, "\n")] = '\0';
```

`fgets()` reads a sentence containing spaces. The next statement removes the newline character.

Step 3: Copy the sentence

```
strcpy(temp, sentence);
```

A copy is created because `strtok()` modifies the string it processes.

Step 4: Separate the words

```
word = strtok(temp, " \t");
```

This extracts the first word. The delimiter string contains a space and a tab.

Step 5: Compare word lengths

```
if ((int)strlen(word) > maxLength)
{
    maxLength = strlen(word);
```

```
    strcpy(longest, word);
}
```

- `strlen()` calculates the length of the current word.

- If the current word is longer, its length is saved in `maxLength`.

- `strcpy()` copies that word into `longest`.

Step 6: Process the remaining words

```
word = strtok(NULL, " \t");
```

This extracts the next word. The loop continues until all words have been checked.

5. Sample Output

```
Enter a sentence: C programming is interesting
Longest word: interesting
```

Question 2: Check Whether One String Is a Rotation of Another

1. Aim

To write a C program using a separate user-defined function to check whether one string is a rotation of another string.

2. Algorithm

1. Start the program.

2. Read two strings from the user.

3. Call the `isRotation()` function.

4. Check whether both strings have equal lengths.

5. If their lengths differ, return false.

6. Concatenate the first string with itself.

7. Search for the second string in the concatenated string using `strstr()`.

8. If the second string is found, the strings are rotations of each other.

9. Otherwise, they are not rotations.

10. Display the result and stop the program.

3. C Program

```
#include <stdio.h>
#include <string.h>

#define SIZE 200

int isRotation(char str1[], char str2[]);

int main()
{
    char str1[SIZE], str2[SIZE];
    int result;

    printf("Enter first string: ");
    fgets(str1, SIZE, stdin);
    str1[strcspn(str1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(str2, SIZE, stdin);
    str2[strcspn(str2, "\n")] = '\0';

    result = isRotation(str1, str2);
```

```
    if (result == 1)
        printf("The strings are rotations of each other.\n");
    else
        printf("The strings are not rotations of each other.
\n");

    return 0;
}

int isRotation(char str1[], char str2[])
{
    char temp[2 * SIZE];

    if (strlen(str1) != strlen(str2))
        return 0;

    strcpy(temp, str1);
    strcat(temp, str1);

    if (strstr(temp, str2) != NULL)
        return 1;

    return 0;
}
```

4. Step-by-step explanation

Step 1: Read two strings

```
fgets(str1, SIZE, stdin);
fgets(str2, SIZE, stdin);
```

These statements read both strings, including spaces.

Step 2: Compare string lengths

```
if (strlen(str1) != strlen(str2))
    return 0;
```

A rotation must have the same length as the original string. If the lengths differ, the strings cannot be rotations.

Step 3: Duplicate the first string

```
strcpy(temp, str1);
strcat(temp, str1);
```

Suppose the first string is `ABCD`.

After these statements, `temp` contains:

```
ABCDABCD
```

Step 4: Search for the second string

```
if (strstr(temp, str2) != NULL)
    return 1;
```

`strstr()` searches for the second string inside the duplicated first string.

For example, `CDAB` occurs inside `ABCDABCD`, so it is a rotation of `ABCD`.

Step 5: Return the result

- `return 1` means the strings are rotations.

- `return 0` means they are not rotations.

5. Sample Output

Example 1 — Rotation:

```
Enter first string: ABCD
Enter second string: CDAB
The strings are rotations of each other.
```

Example 2 — Not a rotation:

```
Enter first string: ABCD
Enter second string: ACBD
The strings are not rotations of each other.
```

Example 3 — Different lengths:

```
Enter first string: ABC
Enter second string: ABCD
The strings are not rotations of each other.
```

6. Combined Menu-Driven Program for Practical Submission

```
#include <stdio.h>
#include <string.h>

#define SIZE 200

void findLongestWord(char sentence[], char longest[]);
int isRotation(char str1[], char str2[]);

int main()
```

```
{
    char sentence[SIZE], longest[SIZE];
    char str1[SIZE], str2[SIZE];
    int choice, result;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Find longest word\n");
        printf("2. Check string rotation\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice)
        {
            case 1:
                printf("Enter a sentence: ");
                fgets(sentence, SIZE, stdin);
                sentence[strcspn(sentence, "\n")] = '\0';

                findLongestWord(sentence, longest);

                if (longest[0] == '\0')
                    printf("No word found.\n");
                else
                    printf("Longest word: %s\n", longest);
                break;

            case 2:
                printf("Enter first string: ");
                fgets(str1, SIZE, stdin);
                str1[strcspn(str1, "\n")] = '\0';

                printf("Enter second string: ");
                fgets(str2, SIZE, stdin);
                str2[strcspn(str2, "\n")] = '\0';

                result = isRotation(str1, str2);

                if (result)
                    printf("Strings are rotations.\n");
                else
                    printf("Strings are not rotations.\n");
```

```
                break;

            case 3:
                printf("Program exited.\n");
                break;

            default:
                printf("Invalid choice!\n");
        }

    } while (choice != 3);

    return 0;
}

void findLongestWord(char sentence[], char longest[])
{
    char temp[SIZE];
    char *word;
    int maxLength = 0;

    strcpy(temp, sentence);
    longest[0] = '\0';

    word = strtok(temp, " \t");

    while (word != NULL)
    {
        if ((int)strlen(word) > maxLength)
        {
            maxLength = strlen(word);
            strcpy(longest, word);
        }

        word = strtok(NULL, " \t");
    }
}

int isRotation(char str1[], char str2[])
{
    char temp[2 * SIZE];

    if (strlen(str1) != strlen(str2))
        return 0;
```

```
    strcpy(temp, str1);
    strcat(temp, str1);

    if (strstr(temp, str2) != NULL)
        return 1;

    return 0;
}
```

7. Sample Output of the Combined Program

```
===== MENU =====
1. Find longest word
2. Check string rotation
3. Exit
Enter your choice: 1

Enter a sentence: C programming is interesting
Longest word: interesting

===== MENU =====
1. Find longest word
2. Check string rotation
3. Exit
Enter your choice: 2

Enter first string: ABCD
Enter second string: CDAB
Strings are rotations.

===== MENU =====
1. Find longest word
2. Check string rotation
3. Exit
Enter your choice: 3
Program exited.
```

8. Result

The menu-driven C program successfully finds the longest word in a sentence and checks whether two strings are rotations of each other using separate user-defined functions.